

# Lab guidance

---

## Hardness of assignments

Each lab task is tagged to indicate roughly how long we expect the task to take:

- Easy: A few hours.
- Moderate: ~ 6 hours (per week).
- Hard: More than 6 hours (per week). If you start late, your solution is unlikely to pass all tests.

Most of the labs require only a modest amount of code (perhaps a few hundred lines per lab part), but can be conceptually difficult and may require a good deal of thought and debugging. Some of the tests are difficult to pass.

Don't start a lab the night before it is due; it's more efficient to do the labs in several sessions spread over multiple days. Tracking down bugs in distributed systems is difficult, because of concurrency, crashes, and an unreliable network.

## Tips

- Do the [Online Go tutorial](#) and consult [Effective Go](#). See [Editors](#) to set up your editor for Go.
- The lab Makefiles are set up to use Go's [race detector](#). Fix any races it reports.
- Advice on [locking](#) in labs.
- Advice on [structuring](#) your Raft lab.
- This [Diagram of Raft interactions](#) may help you understand code flow between different parts of the system.
- Learn about Go's Printf format strings: [Go format strings](#).
- To learn more about git, look at the [Pro Git book](#) or the [git user's manual](#).

## Debugging

---

Efficient debugging takes experience. It helps to be systematic: form a hypothesis about a possible cause of the problem; collect evidence that might be relevant; think about the information you've gathered; repeat as needed. For extended debugging sessions it helps to keep notes, both to accumulate evidence and to remind yourself why you've discarded specific earlier hypotheses.

The most effective debugging technique is often to add print statements to your code, run the test that is failing and collect the print output in a file, and then look through the output file to identify the point at which things start to go wrong. You may need to iterate, adding more print statements as you learn more about what is going wrong.

Concurrency among different peers and among the threads in a single peer can cause actions to be interleaved in unexpected ways. For example, it's quite possible for a Raft peer to be elected leader while the previous leader still thinks it is the leader, or for a leader to send an RPC but receive the reply after it has lost leadership. Adding print statements may help you spot such situations.

Feel free to examine the test code (`mr/mt_test.go`, `raft1/raft_test.go`, &c) to understand what the tests are exploring. You can add print statements to the tests to help you understand what they are doing and why they are failing, but be sure your code passes with the original test code before submitting.

The Raft paper's Figure 2 must be followed fairly exactly. It is easy to miss a condition that Figure 2 says must be checked, or a state change that it says must be made. If you have a bug, re-check that all of your code adheres closely to Figure 2.

As you're writing code (i.e., before you have a bug), it may be worth adding explicit checks for conditions that the code assumes to be true, perhaps using Go's [panic](#). Such checks may help detect situations where later code unwittingly violates the assumptions.

The TAs are happy to help you think about your code during office hours, but you're likely to get the most mileage out of limited office hour time if you've already dug as deep as you can into the situation.

---

---

6.5840 Collaboration policy

## Collaboration Policy

---

You must write all the code you hand in for 6.5840, except for code that we give you as part of assignments. You are not allowed to look at anyone else's solution, and you are not allowed to look at solutions from previous 6.5840 or 6.824 years. You may discuss the assignments with other students, but you may not look at or copy each others' code. The reason for this rule is that we believe you will learn the most by designing and implementing your lab solution yourself.

Please do not publish your code or make it available to current or future 6.5840 students. `github.com` repositories are public by default, so please don't put your code there unless you make the repository private. You may find it convenient to use [MIT's GitHub](#), but be sure to create a private repository.

---

---